

Multihead
Attention

PReLU

ReLU

ReLU6

RReLU

SELU

CELU

GELU

Sigmoid

SiLU

Mish

Softplus

Softshrink

Softsign

Tanh

Tanhshrink

Threshold

GLU

Softmin

Softmax

Softmax2d

LogSoftma
x

AdaptiveL

CrossEntropyLoss

class

```
torch.nn.CrossEntropyLoss(weight,  
size_average=None,  
ignore_index=-100, reduce=None,  
reduction='mean',  
label_smoothing=0.0) \[source\]
```

This criterion computes the cross entropy loss between input logits and target.

It is useful when training a classification problem with C classes. If provided, the optional argument `weight` should be a 1D *Tensor* assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

The *input* is expected to contain the unnormalized logits for each class (which do *not* need to be positive or sum to 1, in general). *input* has to be a *Tensor* of size (C) for unbatched input, $(minibatch, C)$ or

☰ On this page

CrossEntropyLoss

CrossEntropyLoss.forward()

 [Show Source](#)

PyTorch Libraries

[torchao](#) 

[torchrec](#) 

[torchft](#) 

[TorchCodec](#) 

[torchvision](#) 

[ExecuTorch](#) 

[PyTorch on XLA Devices](#) 

logSoftmax
WithLoss
BatchNorm1d

$(minibatch, C, d_1, d_2, \dots, d_K)$
with $K \geq 1$ for the K -dimensional case. The last being useful for higher dimension inputs, such as computing cross entropy loss per-pixel for 2D images.

The *target* that this criterion expects should contain either:

- Class indices in the range $[0, C)$ where C is the number of classes; if *ignore_index* is specified, this loss also accepts this class index (this index may not necessarily be in the class range). The unreduced (i.e. with `reduction` set to `'none'`) loss for this case can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_I\}$$

where x is the input, y is the target, w is the weight, C is the number of classes, and N spans the minibatch dimension as well as $d_1, \dots, d_k$ for the K -dimensional case. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \sum_{n=1}^N \frac{w_n}{\sum_{n=1}^N w_n} \\ \sum_{n=1}^N l_n, \end{cases}$$

Note that this case is equivalent to applying `LogSoftmax` on an input, followed by `NLLLoss`.

- Probabilities for each class; useful when labels beyond a single class per minibatch item are required, such as for blended labels, label smoothing, etc. The unreduced (i.e. with `reduction` set to `'none'`) loss for this case can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_I\}$$

where x is the input, y is the target, w is the weight, C is the number of classes, and N spans the minibatch dimension as well as $d_1, \dots, d_k$ for the K -dimensional case. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \frac{\sum_{n=1}^N l_n}{N}, & \text{if} \\ \sum_{n=1}^N l_n, & \text{..} \end{cases}$$

$$(\sum_{n=1}^N l_n, \quad \text{if}$$

Note

The performance of this criterion is generally better when *target* contains class indices, as this allows for optimized computation. Consider providing *target* as class probabilities only when a single class label per minibatch item is too restrictive.

Parameters:

- **weight** (*Tensor*, optional) – a manual rescaling weight given to each class. If given, has to be a Tensor of size C.
- **size_average** (*bool*, optional) – Deprecated (see `reduction`). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default:

`size_average` is `True`

- **ignore_index** (*int, optional*) – Specifies a target value that is ignored and does not contribute to the input gradient. When `size_average` is `True`, the loss is averaged over non-ignored targets. Note that `ignore_index` is only applicable when the target contains class indices.
- **reduce** (*bool, optional*) – Deprecated (see `reduction`). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`
- **reduction** (*str, optional*) – Specifies the reduction to apply to the output: `'none'` | `'mean'` | `'sum'`. `'none'`: no reduction will be applied, `'mean'`: the weighted mean of the output is taken,

`'sum'`: the output will be

summed. Note:

`size_average` and `reduce`

are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`.

Default: `'mean'`

- **label_smoothing** (*float, optional*) – A float in $[0.0, 1.0]$. Specifies the amount of smoothing when computing the loss, where 0.0 means no smoothing. The targets become a mixture of the original ground truth and a uniform distribution as described in [Rethinking the Inception Architecture for Computer Vision](#). Default: 0.0.

Shape:

- Input: Shape (C) , (N, C) or $(N, C, d_1, d_2, \dots, d_K)$ with $K \geq 1$ in the case of K -dimensional loss.
- Target: If containing class indices, shape $()$, (N) or $(N, d_1, d_2, \dots, d_K)$ with $K \geq 1$ in the case of K -

dimensional loss where

each value should be between $[0, C)$. The target data type is required to be long when using class indices. If containing class probabilities, the target must be the same shape input, and each value should be between $[0, 1]$. This means the target data type is required to be float when using class probabilities. Note that PyTorch does not strictly enforce probability constraints on the class probabilities and that it is the user's responsibility to ensure `target` contains valid probability distributions (see below examples section for more details).

- Output: If reduction is 'none', shape $()$, (N) or $(N, d_1, d_2, \dots, d_K)$ with $K \geq 1$ in the case of K-dimensional loss, depending on the shape of the input. Otherwise, scalar.

where:

C = number of classes
 N = batch size

Examples

```
>>> # Example of target with class probabilities
>>> loss = nn.CrossEntropyLoss()
>>> input = torch.randn(3, 5, device='cuda')
>>> target = torch.empty(3, dtype=torch.float, device='cuda')
>>> output = loss(input, target)
>>> output.backward()
>>>
>>> # Example of target with class indices
>>> input = torch.randn(3, 5, device='cuda')
>>> target = torch.randint(5, (3, ), device='cuda')
>>> output = loss(input, target)
>>> output.backward()
```

Note

When `target` contains class probabilities, it should consist of soft labels—that is, each `target` entry should represent a probability distribution over the possible classes for a given data sample, with individual probabilities between `[0, 1]` and the total distribution summing to 1. This is why the `softmax()` function is

applied to the `target` in the class probabilities example above.

PyTorch does not validate whether the values provided in `target` lie in the range `[0, 1]` or whether the distribution of each data sample sums to `1`. No warning will be raised and it is the user's responsibility to ensure that `target` contains valid probability distributions. Providing arbitrary values may yield misleading loss values and unstable gradients during training.

Examples

```
>>> # Example of target with i
>>> loss = nn.CrossEntropyLoss
>>> torch.manual_seed(283)
>>> input = torch.randn(3, 5,
>>> target = torch.randn(3, 5)
>>> # Provided target class pr
>>> target
tensor([[ 0.7105,  0.4446,  2.
         [-1.0496, -0.2753, -0.
         [ 0.7551,  0.1003,  1.
>>> # Provided target class pr
>>> target.sum(axis=1)
tensor([2.8444, 0.2462, 0.8873
>>> # No error message and pos
>>> loss(input, target).item()
4.6379876136779785
>>>
>>> # Example of target with c
>>> # Use .softmax() to ensure
>>> target_new = target.softma
>>> # New target class probabi
>>> target_new
tensor([[0.1559, 0.1195, 0.583
        [0.0496, 0.1075, 0.099
        [0.2607, 0.1355, 0.471
>>> # New target class probabi
>>> target_new.sum(axis=1)
tensor([1.0000, 1.0000, 1.0000
>>> loss(input, target_new).it
2.55349063873291
```

forward(*input*, *target*)

Runs the forward [\[source\]](#)
pass.

Return type:

Tensor

Rate this Page ★ ★ ★ ★ ★

Send Feedback

< Previous
[MSELoss](#)

Next
[CTCLoss](#) >

© Copyright PyTorch Contributors.

Docs

Access comprehensive
developer
documentation for
PyTorch

[View Docs](#)

Tutorials

Get in-depth tutorials for
beginners and advanced
developers

[View Tutorials](#)

Resources

Find development
resources and get your
questions answered

[View Resources](#)

Stay in touch for updates, event info,
and the latest news

First Name^{*}

Last Name^{*}

Email^{*}

Select Cou

SUBMIT

in



[Policies](#)

[Trademark Usage](#)

[Privacy Policy](#)

